

Batch Transfer for Offline Corporate Clients

Product Design - Design System - B2B Banking - 2026

A forced-timeline redesign that shipped because the system underneath it had already been built for reuse.

Context

Some corporate clients would not transact online. For security policies, internal approval rules, or institutional caution, they continued to use an offline process: they brought physical cheques and an Excel payment file into a branch, and a bank employee processed the batch manually through an internal tool.

The bank already had a legacy tool for this workflow, but it created friction in exactly the moment where reliability mattered most. It was slow, error-prone, and unforgiving: one small mistake could turn a routine salary batch into hours of manual correction.

We were asked to replace it on a fixed timeline.

The Problem

The legacy system treated the uploaded payment file as one block. If a single transaction in a batch of 200 had a problem - for example, a wrong account number, a bad IBAN, or a recipient that could not be verified - the entire file failed.

The employee only found out at the end. From there, they had to search through the Excel file, identify the broken row, fix it, and upload the full batch again. There was no meaningful validation upfront, no row-level feedback, and no clear way to verify recipients before submission.

For the employee, the problem was not simply that an error happened. It was that the system made the error expensive to find. One typo could block a full batch and waste an afternoon.

My Role

I worked with one other designer. We did the research together, visiting branches in person and speaking directly with the employees who processed these files in real operational conditions.

After research, I owned the product design: the flow, wireframes, stakeholder prototype, and final UI. My teammate and I stayed aligned on direction, but the interaction and interface decisions for the batch transfer experience were mine.

The Constraint That Shaped the Work

The timeline was fixed. The requirement came from the bank with a hard date, which meant the ideal process - research, design, full usability testing, iteration, and then launch - was not realistic.

The usual risk in that situation is that quality gets traded for speed. In this project, speed came from a different place: reuse.

Before this project, I had already built a design system for another banking product. I had designed it beyond the immediate product need, with shared components, tokens, states, and patterns that could support other internal tools across the bank. That decision mattered here. Instead of

designing every screen from zero, I could assemble the new flow from an existing foundation and focus my attention on the workflow logic, validation model, and edge cases.

The speed did not come from skipping design work. It came from systems work that had already been done on purpose.

Core Product Decision: Kill the All-or-Nothing Failure Model

The most important decision was to stop treating the batch as a single pass/fail object.

In the redesigned flow, each transaction is checked independently. The system validates rows before final submission, verifies recipient account information where possible, and gives the employee a clear review state before the batch is finalized.

Instead of discovering a failure after the full batch was processed, the employee sees exactly which rows have problems and why. Valid rows are no longer made invisible by one invalid row. The broken row becomes something specific to correct, not a reason to restart the whole afternoon.

That sounds small, but in this workflow it changes the cost of an error. The old system turned one mistake into a failed batch. The new system turns one mistake into one row that needs attention.

Design Strategy

I designed the flow around four principles:

- Surface problems before submission. Employees should not wait until the end to learn that a file has failed.
- Make errors row-level, not file-level. A problem should point to the exact transaction that needs attention.
- Support verification, not guesswork. Recipient and account checks should help employees trust the batch before they submit it.
- Reuse existing system patterns wherever possible. Under a forced timeline, consistency and speed had to come from shared components, not one-off UI decisions.

The redesigned flow moved through upload, validation, row-level review, correction decision, and final submission. The review step became the key safety gate: it gave employees enough information to decide whether to proceed, cancel, fix the source file, or re-upload with confidence.

Process

We started with in-person branch research. That mattered because this was not a workflow we could understand from requirements alone. The pain was operational: employees were under time pressure, working with real client files, and trying to avoid mistakes in a high-trust banking context.

From there, I created wireframes and a prototype to get stakeholder alignment before UI production. The prototype helped lock the structure of the flow early, so major disagreements could happen before engineering effort was spent.

Because the timeline did not allow a full usability testing cycle, we used a controlled rollout instead. The product launched first in a single branch, where real employees used it on real batches and gave feedback. We iterated based on that feedback before expanding the rollout.

I would not describe that as a perfect substitute for usability testing. It was the most realistic substitute available under the constraint, and it gave us feedback from actual usage rather than simulated tasks.

Outcome

The redesigned batch transfer tool launched through a controlled branch rollout. The branch used it on live payment batches, gave feedback, and we made adjustments before broader expansion.

I do not have instrumented post-launch metrics. The bank did not share before/after data with us, and I do not want to turn observation into fake precision. What I can say honestly is that the core failure mode of the old system - one invalid row rejecting the entire batch without useful feedback - was removed from the workflow.

If I had access to operational data, I would measure failed-batch rate, number of re-uploads per batch, time to identify a bad row, time to complete a corrected batch, and support requests from branches. Those are the metrics that would prove whether the redesign reduced operational waste in the way it was intended to.

Reflection

The lesson from this project is not only about batch transfer. It is about the value of design systems as operational infrastructure.

A forced timeline is usually where product quality gets fragile. This project held together because the design system had been built for reuse before the pressure arrived. Shared components, states, and patterns meant I could spend less time recreating UI foundations and more time solving the actual workflow problem.

Systems work is often invisible until the moment it saves time. This was that moment.

What I would do differently: I would push for at least one structured usability session before launch, even on a compressed timeline. The controlled rollout caught issues, but it caught them in production. Some of those issues would have been cheaper to find earlier.